Numeric functions

Common Scalar Numeric Functions

Scalar functions return a single value for each row they are applied to.

Function	Description	Example SQL Query	Example Output
`ABS(x)`	Returns the absolute value of `x`.	`SELECT ABS(-10);`	`10`
`ROUND(x, d)`	Rounds `x` to `d` number of decimal places. If `d` is omitted, it rounds to the nearest integer.	`SELECT ROUND(15.193, 1);`	`15.2`
`FLOOR(x)`	Returns the largest integer value that is not greater than `x` (rounds down).	`SELECT FLOOR(15.9);`	`15`
`CEIL(x)` / `CEILING(x)`	Returns the smallest integer value that is not less than `x` (rounds up).	`SELECT CEIL(15.1);`	`16`
`TRUNCATE(x, d)`	Truncates `x` to `d` number of decimal places without rounding.	`SELECT TRUNCATE(15.193, 1);`	`15.1`
`MOD(x, y)`	Returns the remainder of `x` divided by `y` (modulo operator).	`SELECT MOD(11, 4);`	`3`
`POWER(x, y)`	Returns the value of `x` raised to the power of `y`.	`SELECT POWER(3, 2);`	`9`
`SQRT(x)`	Returns the non-negative square root of `x`.	`SELECT SQRT(25);`	`5`

`SIGN(x)`	Returns the sign of `x`: -1 for negative, 1 for positive, and 0 for zero.	`SELECT SIGN(-15);`	`-1`
`RAND()`	Returns a random floating-point value between 0 (inclusive) and 1 (exclusive).	`SELECT RAND();`	`0.4578...` (varies)

Common Aggregate Numeric Functions

Aggregate functions perform calculations on a set of rows and return a single value.

Function	Description	Example SQL Query (using a conceptual **`products`** table)	Example Output
`AVG(column)`	Returns the average value of a column.	`SELECT AVG(price) FROM products;`	`35.50`
`SUM(column)`	Calculates the sum of values in a column.	`SELECT SUM(quantity) FROM products;`	`150`
`MIN(column)`	Returns the minimum value in a column.	`SELECT MIN(price) FROM products;`	`10.00`
`MAX(column)`	Returns the maximum value in a column.	`SELECT MAX(price) FROM products;`	`99.99`
`COUNT(column)`	Returns the number of rows (usually used with other data types but functions numerically).	`SELECT COUNT(product_id) FROM products;`	

`FORMAT()` function

The MySQL `FORMAT()` function is used to format a numeric value as a string with a specified number of decimal places, including commas for thousand separators. It also rounds the number to the specified decimal place.

Syntax

`FORMAT(X, D, [locale])`

- **X:** The number to format.
- **D:** The number of decimal places to keep.
- **locale (Optional):** Specifies the regional formatting (e.g., decimal point character and thousands separator). If omitted, it defaults to `'en_US'`.

Examples and Outputs

Example Query	Purpose	Output (String)
`SELECT FORMAT(12345.6789, 2);`	Standard format with 2 decimals	`'12,345.68'`
`SELECT FORMAT(1234567.8, 0);`	No decimal places	`'1,234,568'`
`SELECT FORMAT(12345.6789, 3, 'de_DE');`	German locale (dots for thousands, comma for decimals)	`'12.345,679'`
`SELECT FORMAT(5, 2);`	Padding a small number with zeros	`'5.00'`

Key Considerations for 2025

- **Return Type:** The output is always a **string**, not a numeric type.
- **Sorting:** Because it returns a string, using `ORDER BY` on a formatted column will result in alphabetical sorting (e.g., "1,000" comes before "200") rather than numeric.
- **Rounding:** The function performs standard rounding on the last digit specified.

- **Supported Types:** It works with `DECIMAL`, `FLOAT`, and `DOUBLE` data types.

IF() Function

The MySQL `IF()` function is a control flow function used to return one value if a condition is true and another value if it is false.

Syntax

```
IF(condition, value_if_true, value_if_false)
```

- **Condition:** The expression to evaluate.
- **value_if_true:** Returned if the condition is **TRUE** (not zero and not NULL).
- **value_if_false:** Returned if the condition is **FALSE** or NULL.

Examples and Outputs

Example Query	Purpose	Output
`SELECT IF(100 > 50, 'Yes', 'No');`	Simple numeric comparison	`'Yes'`
`SELECT IF(STRCMP('a', 'b') = 0, 'Match', 'No Match');`	Compare two strings	`'No Match'`
`SELECT IF(Quantity > 10, 'High', 'Low') FROM Orders;`	Categorize table data by column	`'High'` or `'Low'`
`SELECT IF(price IS NULL, 0, price) AS FinalPrice;`	Handle NULL values	`0` or `price`

Key Applications in 2025

- **Data Categorization:** Use `IF()` in a `SELECT` statement to create virtual labels for data ranges (e.g., labeling prices over $100 as "Expensive").
- **Conditional Aggregation:** Combine with aggregate functions like `SUM()` to count specific occurrences.
  - *Example:* `SELECT SUM(IF(status = 'Completed', 1, 0)) AS TotalCompleted;`
- **Replacing NULLs:** While `IFNULL()` or `COALESCE()` are more common for null handling, `IF(col IS NULL, 'N/A', col)` provides similar flexibility.

NULLIF() and COALESCE()

In MySQL, `NULLIF()` and `COALESCE()` are powerful tools for managing `NULL` values and specific data conditions.

1. NULLIF() Function

The `NULLIF()` function compares two expressions. If they are **equal**, it returns `NULL`. If they are **not equal**, it returns the first expression.

- **Syntax:** `NULLIF(expression1, expression2)`
- **Common Use Case:** Preventing "division by zero" errors by converting a `0` divisor into `NULL`.

Example Query	Purpose	Output
`SELECT NULLIF(5, 5);`	Same values	`NULL`
`SELECT NULLIF(10, 5);`	Different values	`10`
`SELECT 100 / NULLIF(0, 0);`	Avoid division by zero	`NULL`
`SELECT NULLIF('A', 'B');`	Comparing strings	`'A'`

---

2. COALESCE() Function

The `COALESCE()` function returns the **first non-NULL value** from a list of arguments. If all arguments in the list are `NULL`, it returns `NULL`.

- **Syntax:** `COALESCE(value1, value2, ..., valueN)`
- **Common Use Case:** Providing a default "fallback" value for optional columns or missing data.

Example Query	Purpose	Output
`SELECT COALESCE(NULL, NULL, 10, 20);`	Find first non-NULL	`10`
`SELECT COALESCE(NULL, 'Default');`	Provide a fallback	`'Default'`
`SELECT COALESCE(phone, email, 'No Contact');`	Multi-column fallback	First valid contact
`SELECT COALESCE(NULL, NULL);`	All arguments NULL	`NULL`

Comparison Table (Summary)

Feature	NULLIF()	COALESCE()
Primary Goal	Returns `NULL` if values match.	Returns the first available data.
Arguments	Always exactly **two**.	**One or more** (multiple possible).
Logical Inverse	Often considered the "inverse" of `COALESCE`.	Used to "fill in" gaps in data.

Typical Usage

```
SELECT 10 / NULLIF(qty, 0)
```

```
SELECT COALESCE(price, 0)
```